Telegram Widgets

Fonte: https://core.telegram.org/bots/widgets?utm_source=chatgpt.com · Extraído em 10 de novembro de 2026

Bots can send **widgets**: small interactive blocks drawn right inside a message. A widget is one self-contained HTML document with inline CSS and JavaScript, at most 32 KB of HTML and 4 KB after compression. Telegram apps run it on the user's device in a sandbox with **no network access at all**, draw it in the style of the chat, and keep it running while it is on screen: a poll whose bars move as people vote, a price ticker, a board game for two in a group, a promo code that copies on a tap.

A widget is **fixed code plus changeable state**. The code, its HTML, is set when the widget is sent and never changes. What changes is the widget's state, and only the bot writes it: the **chat state**, tied to one message and seen only in that chat, and the **shared state**, seen wherever the widget is shown. The widget reads its state and redraws when it changes. It cannot change the state itself, but it can ask the bot with small requests, the way a callback button does. Widgets live in the private chat with the bot, in groups, in messages sent via the bot in inline mode, and in forwards to any chat. Currently, in the production environment, widgets work only in the private chat with the bot. The rest works on the test server. A widget can also be a **shared widget**: one widget on the server, like a poll, that any number of messages show, with one shared state for all of them.

This page is a guide to designing and implementing widgets, followed by the reference of the widget's JavaScript object `TelegramWidget`. The Bot API methods and types for widgets are in the Bot API reference.

Introducing widgets

Code and state

A widget is fixed code plus changeable state.

- **The code** (the HTML with its CSS and JavaScript) is set when the widget is sent and never changes, in a message or in a shared widget. No method replaces it. For a new layout, send a new widget, and old messages keep the old one. As with any media, the caption can be edited.
- **The state** is what changes, and only the bot writes it. There are two kinds:
  - **chat state** (`chat_data`): tied to one message, seen only in that chat (shared by all members in a group), never carried by a forward. A game in progress, whose turn it is.
  - **shared state** (`public_data`): seen wherever the widget is shown. A static widget keeps it in the message, and a forward gets a copy as of the moment of forwarding. A

shared widget (one object with an identifier, like a poll) has one shared state for all its messages and forwards. Poll results, a price.

- **The widget** reads its state, learns about changes through the `chatDataChanged` and `publicDataChanged` events, and cannot change its state itself: it can only ask the bot, with a request.

Details: The widget's state.

What a widget is

- **One HTML document.** Inline `<style>` and `<script>`, images only as `data:` URLs or inline SVG. At most **32768 bytes** of UTF-8 HTML and at most **4096 bytes** after the server compresses it with zlib at level 9. The bot sends plain HTML, and the server checks and packs it. A bare fragment without `<html>` and `<body>` works too.
- **Static or shared.** A static widget keeps its code and state in the message, and a forward copies them as they are at that moment. A shared widget is one object on the server holding the code, the size, the shared state, the refresh period and the request flag. Any number of messages show it by its identifier, and a change of its shared state reaches all of them.
- **Sandboxed.** The widget's JavaScript runs in QuickJS, a small JavaScript engine, never in a WebView or browser context. The client checks every change the widget makes to its page against the widget profile and draws only what passes. The same widget looks the same on iOS, Android, macOS, Telegram Desktop and the web.
- **Offline.** `fetch`, `XMLHttpRequest`, `WebSocket` and every other network API throw `NetworkDisabledError`. External scripts, stylesheets, fonts and images are never loaded, and the widget cannot navigate. No library is bundled.
- **Native.** The chat's palette is available as CSS variables, and the client's own default styles make a widget without any CSS look like an ordinary message, with buttons in the client's style.
- **Connected through the bot only.** The widget's one way out is a request to the bot of up to 64 bytes, which arrives as a widget_query update. The bot's answer goes back to the widget that asked, and only the bot changes the widget's state. The messenger, not the sandbox, carries the traffic.

Where widgets live

Where	How it gets there	Requests (for a widget sent with `allow_queries`)	Chat state	In production
The private chat with the bot	sendWidget	on a tap, and in the background	the user's	Yes
A basic group or supergroup the bot is a member of	sendWidget	on a tap	one for all members	Test server only
Any chat except channels, sent via the bot	inline mode	on a tap	per sent message	Test server only

Where	How it gets there	Requests (for a widget sent with `allow_queries`)	Chat state	In production
A forward, in any chat (channels included)	forwarded by users	on a tap, only for a shared widget	never	Test server only

Widgets cannot be posted in channels, directly or via the bot, but users can forward widgets there. Apps that do not support widgets show the message as unsupported media with its caption.

Availability

Widgets are rolling out in steps. Everything on this page works on the test server. Currently, in the production environment, widgets work only in the private chat with the bot.

Feature	Test server	Production
Widgets in the private chat with the bot, with requests and the wallet	Yes	Yes
Forwarding widget messages	Yes	No
Widgets in groups	Yes	No
Widgets in inline mode	Yes	No

- **On the test server** every bot can build and test the full set: groups, inline mode and forwards.
- **In the production environment**, a bot sends widgets only to private chats with users, and users cannot forward its widget messages. A widget in a private chat that cannot be forwarded has no forwarded copies: design it for that chat alone.

The sections below describe the full set. Each section that is not yet in production says so.

Widgets and Mini Apps

Widgets are not small Mini Apps. A Mini App is a web page you host, opened on demand. A widget is part of the message, always visible as the chat scrolls, and has no network access.

Feature	Mini Apps	Widgets
Where	a web view opened from a button, the menu or a link	inside the message bubble
Code	a web page on your server, loaded at launch	at most 32 KB of HTML inside the message, fixed once created
Engine	the platform's WebView	the QuickJS sandbox, with the page changes checked and drawn by the Telegram app
Network	the web	none

Feature	Mini Apps	Widgets
Talking to the bot	your own backend, with signed `initData`	`TelegramWidget.request()`, delivered by Telegram as a `widget_query` or `shared_widget_query` update
State	your backend, `CloudStorage`, `DeviceStorage`	the chat state and the shared state, written by the bot, and Web Storage per message on the device
Who is looking	`initData.user`	`TelegramWidget.viewer` (id, name, username, Premium), only where the widget may send requests
Input	everything, keyboard included	taps, pointer events, horizontal drags and holds, but no keyboard

Where the two APIs overlap, `TelegramWidget` uses the names of `Telegram.WebApp` (`themeParams`, `colorScheme`, `onEvent`, `openLink`, `openTelegramLink`, `HapticFeedback`, `ready`, and `viewer` shaped like `WebAppUser`), so code and habits carry over. For anything larger than a widget, link to your bot or Mini App with `openTelegramLink`.

Designing widgets

Native look

Widgets are drawn on the message bubble, in the chat's colors and text style. Use them instead of your own:

- **The bubble is the canvas.** The widget fills the bubble with no padding of its own, and the bubble clips it at its rounded corners. The root element has no background: the bubble's color (`--tg-theme-bg-color`) shows through. A background you set on `html` or `body` paints the whole bubble, up to the rounded edges.
- **Default styles.** The app inserts its own default stylesheet beneath yours. Text gets the chat's message color, size and line height. The `body` margin equals the app's text padding (about `8px 12px`), so text starts where a message's text starts. Headings, paragraphs, lists, tables (with separator lines), `pre`, `code`, quotes, links and figure captions get native sizes and margins. Any rule of yours wins over these defaults, whatever its specificity. `body { margin: 0 }` goes edge to edge.
- **Buttons.** A `<button>` you do not style is drawn as the client's own button, with a pressed state (`:active`) and a disabled state (`[disabled]`). Your CSS overrides it. It is the only form control: inputs, selects and text areas are removed.
- **The palette is the bubble's.** A widget sent by the bot sits in an incoming bubble. A widget the user sent via the bot or forwarded sits in their outgoing bubble, with that bubble's colors. Never hard-code colors that must contrast with the bubble, use the variables instead.
- **The time sits on the widget** when the message has no caption below it: see The time in the corner.

A widget without a single line of CSS already looks native:

```
<h3>Order #1042</h3>
<p>2 × Margherita, 1 × Lemonade</p>
<p><strong>Total: $23.50</strong></p>
<button
onclick="TelegramWidget.openLink('https://example.com/orders/1042')">Track the
order</button>
```

The palette is on the root element as CSS variables and in JavaScript as `TelegramWidget.themeParams`:

Variable	Use
`--tg-theme-bg-color`	the bubble's color, behind the widget
`--tg-theme-text-color`	message text
`--tg-theme-hint-color`	secondary text: time, meta
`--tg-theme-link-color`	links
`--tg-theme-button-color`, `--tg-theme-button-text-color`	the client's primary button
`--tg-theme-secondary-bg-color`	a second surface on the bubble (quotes, inline buttons)
`--tg-theme-accent-text-color`, `--tg-theme-subtitle-text-color`, `--tg-theme-destructive-text-color`	accent text, subtitles, destructive actions
`--tg-theme-section-separator-color`	separator lines (the defaults draw table lines and `<hr>` with it)

The full list, with the text and button tokens, is in CSS variables.

Color schemes

The chat's color scheme drives everything that depends on light or dark: `@media (prefers-color-scheme: dark)`, `light-dark()` and `color-scheme: light dark` follow the **chat** theme, not the system, and switch live. In JavaScript, read `TelegramWidget.colorScheme` and listen for `themeChanged`. The variables change at the same moment. Test every widget in both schemes and in an outgoing bubble.

Sizes

- **Width.** A bubble is 200-600 px wide, and phones lay it out at about 320-430 px. Lay out with `%`, flex or grid. `@media (min-width / max-width)` and `vw` refer to the bubble's width. Height-relative units (`vh` and the like) are not supported.
- **Height.** With an automatic height the bubble takes the widget's content height, from 50 to 600 px. Content beyond the bubble's height is **clipped**: nothing scrolls inside a widget, and there is no fullscreen. Keep widgets comfortably below the cap: under about 560 px at a 320 px width.

- **Size anchors.** `width` and `height` in sendWidget tell the app the size before the widget has loaded, in logical pixels (CSS px, points, dp). `0` means automatic, for each axis separately. A shared widget gets them when it is created, the same in every message that shows it.

`width`	`height`	**Before the widget loads**	**After its first render**
0	0	a placeholder at the bubble's width	the content height, 50-600 px, remembered per message
0	H	the bubble's width, height clamp(H, 50, 600)	unchanged
W	0	width clamp(W, 200, the bubble's maximum), a placeholder height	the content height, 50-600 px
W	H	a fixed widget: clamp(W, 200, max) × clamp(H, 50, 600)	unchanged

Resize anchor

When a widget's height changes, the app keeps one edge of the bubble in place on screen. By default the top stays and the bubble grows downward. This is right for an FAQ whose answers expand, “show more” or an order summary that opens. A widget that grows at its bottom, such as a chat log or a support thread with quick replies under the last message, calls `TelegramWidget.setResizeAnchor('bottom')`: the bottom edge stays and the content above moves. Call it before the change, or at the start of the script for a widget that always grows one way. It applies until called again. Apps whose framework predates it lack the method:

```
if (typeof TelegramWidget.setResizeAnchor === 'function')
TelegramWidget.setResizeAnchor('bottom');
```

The time in the corner

A widget message with **no caption below the widget** (no caption, or a caption shown above it with `show_caption_above_media`) gets its time and status (the ticks, “edited”) drawn by the app **over the widget's bottom corner**, on a backing, as over a photo: the right corner, or the left one in a right-to-left interface. Taps on the time are the app's: put nothing a user taps under it. With a caption below the widget, the time is in the caption's line and nothing covers the widget.

The widget gets the band the way a web page gets a phone's notch:

```
/* the band the time is drawn in: keep it clear at the bottom */
.footer { padding-bottom: max(8px, env(safe-area-inset-bottom)); }
/* only the corner of the band is covered: content may run into the band on the
other side */
```

```
.footer { padding-right: max(12px, var(--tg-content-safe-area-corner-right));
          padding-left: max(12px, var(--tg-content-safe-area-corner-left)); }
```

- `env(safe-area-inset-bottom)` (also `var(--tg-content-safe-area-inset-bottom)`, the Mini Apps name) is the height of the band, 0 with a caption below. The other insets are 0. `--tg-content-safe-area-corner-right` and `-left` are the width of the covered corner on that side, and the other one is 0.
- The default styles add the band to the body's bottom padding, so a widget without CSS, or one that sets only `body { margin: 0 }`, stays clear of the time. A widget that pads for the band itself sets `body { padding: 0 }` too, or it gets two bands.
- In JavaScript: `TelegramWidget.contentSafeAreaInset` has the same numbers (a canvas widget lays out its own controls with them), and `contentSafeAreaChanged` fires when they change: when an edit adds, removes or moves the caption, or the “edited” mark widens the time. They never change from frame to frame.

Gestures

A widget shares its bubble with the chat's own gestures: the message menu on a hold, swipe-to-reply, the list's scroll. One rule, the same in every Telegram app, decides who gets a press:

- **Taps and double taps over the widget are always the widget's**, on every element. A double tap arrives as two `click`s and then a `dblclick`.
- **Vertical drags are always the chat's**: the chat scrolls over every widget. The widget gets `pointercancel`.
- **Holds and horizontal drags** are claimed per element, when the finger goes down, by CSS `touch-action` on the element or any ancestor:

Over an element with	Hold	Horizontal drag	Vertical drag
`touch-action: auto` (the default)	the chat: message menu	the chat: swipe-to-reply	the chat scrolls
`touch-action: pan-y`	the chat: message menu	**the widget**	the chat scrolls
`touch-action: none`	**the widget** (`contextmenu`)	**the widget**	the chat scrolls

`pan-x` claims nothing. The first move past 10 px decides for the whole press. A drag that starts sideways on a claimed element stays the widget's on both axes until the finger lifts, and one that starts up or down belongs to the chat. When the chat takes a press, the widget gets `pointercancel` and no `click`, so undo whatever `pointerdown` started. A hold is 500 ms within 10 px. A double tap is two taps within 300 ms and 40 px. The leading 20 px of the widget (the right edge in right-to-left interfaces) never take a claim, so the system's back gesture keeps working there. Keep controls away from that edge. With a mouse, a right click follows the same claim, and clicks, holds and drags are the widget's.

Rule of thumb: a button you hold, a game control or a canvas: `touch-action: none` on it. A horizontal slider, carousel or paddle: `touch-action: pan-y`. Everything else stays `auto`, so people keep the menu and reply over the rest of the widget. Nothing can be dragged vertically:

no vertical sliders, wheel pickers, up/down swipes, two-axis joysticks, drawing or map panning. Turn such controls horizontal or use buttons.

Text selection

Text in a widget is not selectable unless you allow it. Add `user-select: text` (and `-webkit-user-select: text`) to what a user may want to copy, such as an address or a code. Children inherit it, and `user-select: none` turns it off again below. Selection starts the way it does in an ordinary message (on touch, from the message menu), and copying uses the app's own command. The widget cannot read the selection. For a one-tap copy, use `copyText`.

Design guidelines

- **Draw the first state at once.** Render synchronously from the state the widget already has. There is nothing to load.
- **Always send a caption.** It is the text in the chat list, notifications and search, and what apps without widget support show instead of the widget.
- **Use the theme.** Variables instead of colors, `em` instead of fixed text sizes, system fonts only (`system-ui`, `sans-serif`, `serif`, `monospace`, `ui-*`).
- **Stay small.** The limit is 4096 bytes compressed, counted as the server counts. `data:` images barely compress, so prefer SVG and CSS.
- **Design for every place the message can go.** A forward has no chat state and may not be able to ask the bot, and a preview in the inline results takes no taps. Show something useful in each. A forward of a static widget never changes, so draw it as a snapshot ("Cooking, as of 19:40"), not as progress the reader would wait for.
- **Mute with color, not transparency.** Up to 16 elements at once may have a blurred shadow, a `filter`, a `backdrop-filter` or `opacity` below 1 (Limits). Grey text and small details with `--tg-theme-hint-color`, `rgba()` or `fill-opacity` in SVG: it looks the same and costs nothing.
- **Show progress and every outcome** of a request: "Sending…", the bot's answer, and each error code.
- **Treat the code and the shared state as public.** Forwards carry them to any chat. In a group, the chat state is visible to every member. Never put secrets in a widget.
- **Respect motion settings.** `prefers-reduced-motion` is supported. Animate with transitions, keyframes or `requestAnimationFrame`, and never read layout in every frame.
- **Do not rely on hover** (`:hover` exists only with a mouse) or on the keyboard (there is none).
- Widgets have no accessibility semantics in this version (`aria-*` attributes are kept but do nothing): put what matters in the caption too.

Implementing widgets

Sending a widget

Use sendWidget to send a new widget. `html` is its source as a string of UTF-8 text. It must not contain the NUL character (U+0000), or the server refuses it with `INPUT_MEDIA_WIDGET_HTML_INVALID`. The server compresses the HTML and refuses anything over the limits (`INPUT_MEDIA_WIDGET_TOO_LARGE`). Use sendSharedWidget to send an existing shared widget again by its identifier, the way a photo is sent again by its `file_id`.

```
POST https://api.telegram.org/bot<token>/sendWidget
{
  "chat_id": 111111111,
  "html": "<h3>Order #1042</h3><p>2 × Margherita, 1 × Lemonade</p>…",
  "caption": "Order #1042: $23.50"
}
```

On success the sent Message is returned, with its widget: the HTML, the size anchors, the current state and, for a shared widget, its `id`.

With `sendWidget`, the bot makes two choices, kept with the widget and carried by its forwards:

Parameter	What it does
`allow_queries`	The widget may send requests to the bot. Without it, every request to this widget is refused, everywhere. Allow requests only if the widget needs to talk to the bot: a ticker or a widget that only shows its state needs none.
`is_shared`	Creates a shared widget: one object for the original, all its forwards and every message later sent with `sendSharedWidget`. Its forwards follow its shared state, and with `allow_queries` they may send requests too, on a tap (forwards: test server only). A ticker without requests is `is_shared` without `allow_queries`.

Without `is_shared` the widget is **static**: everything stays in the message. Also pass the widget's initial state, `public_data` and `chat_data`, and the size anchors `width` and `height` (Sizes) when you know them. The usual message parameters work as for a photo: a caption with its entities (and `show_caption_above_media`, which leaves the time on the widget), an inline keyboard under the widget, a reply, a forum topic, `protect_content`.

The caption can be changed later with editMessageCaption, as for any media. The widget itself cannot be replaced: no method changes the HTML of a sent widget.

The widget's state

A widget is fixed code plus changeable state. The code never changes after sending, and the state is what the bot changes. Only the bot writes the state: when it sends the widget, and later with the methods that change it, never inside an answer to a request. The widget reads its state

as `TelegramWidget.chatData` and `TelegramWidget.publicData` and cannot change it itself. It can only ask the bot.

Property	**Chat state (`chat_data`)**	**Shared state (`public_data`)**
Tied to	one original message: in the bot's private chat with the user, in a group (shared by all its members), or sent via the bot	a static widget's message, or a shared widget (every message and forward that shows it)
Seen	only in that chat	wherever the widget is shown
In a forward	never	a copy as of the moment of forwarding for a static widget, the one current shared state for a shared widget
Examples	a game in progress, whose turn it is, a code this user redeemed	poll results, a price, a score
Changed by	editMessageWidgetData on the original	editMessageWidgetData on an original, or editSharedWidgetData by the shared widget's identifier
Reaches open widgets	at once, pushed by the server	pushed with the message for a static widget. For a shared widget, polled at its refresh period while a widget that called `watchPublicData()` is on screen, otherwise with the message
In the widget	`chatData`, event `chatDataChanged`	`publicData`, event `publicDataChanged`

The format is the same everywhere (sending, inline results, changes):

- The chat state and the shared state are each a text of up to **1024 bytes** (bytes of UTF-8, not characters: 512 Cyrillic letters fill it), often JSON. The server does not interpret it.
- At sending, a field not passed means no state. In a change, a field not passed stays as it is, and an empty text clears it. The widget never sees an empty text: a cleared or absent state is `null`.

When the bot changes the state, a running widget gets `chatDataChanged` or `publicDataChanged` with the new state already in place (for a shared widget's shared state, only if the widget called `watchPublicData()`). Nothing restarts and the page keeps what it has drawn, so redraw from the state. A change that touches both fires both, `chatDataChanged` first. A change that leaves one of them as it was fires nothing for it.

```
const W = window.TelegramWidget;
function render() {
  let game = null;
  try { game = JSON.parse(W.chatData); } catch (e) {}
  // … draw from the chat state (null: no state yet)
}
```

```
W.onEvent('chatDataChanged', render);
render();
```

Static and shared widgets

Property	Static widget	Shared widget
Created by	`sendWidget` without `is_shared`	the first `sendWidget` (or inline send) with `is_shared`
Code, size, request flag	in the message	in the shared widget, the same in every message that shows it
Shared state (`public_data`)	in the message. A forward gets a copy as of the moment of forwarding, and a change reaches only the original	in the shared widget: one shared state for every message and forward, changed for all of them at once
Chat state (`chat_data`)	per message	per message, set at each send
Shown again elsewhere	by forwarding (a frozen copy)	by forwarding, or with sendSharedWidget / an inline result by its identifier
Requests from forwards	never	on a tap, if it was sent with `allow_queries`. They arrive as shared_widget_query
Identifier	none	`widget.id` in every message that shows it, for everyone including the bot, and `widget_id` in the methods and updates

A shared widget is like a poll: one object that any number of messages refer to. There is no separate method to create one: **the first send creates it.** To get its identifier in advance, to send the widget to many chats or to offer it in inline mode, send it once with `is_shared` to any chat (for example your own chat with the bot) and read `widget.id` from the returned message.

- **One widget, one behavior.** Every message showing a shared widget has the same HTML, size and request flag, as one poll cannot be anonymous in one message and public in another. Only the chat state belongs to each message.
- **Permanent.** A shared widget outlives every message that shows it: its shared state stays, editSharedWidgetData edits it by its identifier after every carrier is deleted, and requests from live forwards keep arriving.
- **Immutable code.** For a new layout, send a new widget with `sendWidget`, which makes a new shared widget. Messages that show the old one keep it.

Send	Use case	Shared state	Forwards may ask the bot	Change the shared state
`sendWidget`	a widget with state	in the message, forwards get a copy	No	by the original's coordinates, only the original changes

Send	Use case	Shared state	Forwards may ask the bot	Change the shared state
`sendWidget` + `is_shared`	a ticker	a new shared widget	No, forwards follow it	by `widget_id` or by any original's coordinates
`sendWidget` + `is_shared` + `allow_queries`	live forwards (a poll)	a new shared widget	Yes, on a tap	by `widget_id` or by any original's coordinates
`sendSharedWidget`	send an existing shared widget again	the shared widget's, unchanged by the send	per the shared widget's flag	by `widget_id` or by any original's coordinates

Freshness. An edit of a shared widget is visible at once to every read: opening the chat or scrolling to the message gives the current shared state. A widget already on screen gets a new shared state only if it asks for it with `TelegramWidget.watchPublicData()`. While at least one widget that called it is on screen, the app polls that shared widget (one poll per shared widget, whatever the number of messages showing it) at the shared widget's refresh period, and every widget on screen showing it gets `publicDataChanged`. A widget that does not watch gets its shared state only with the message. `unwatchPublicData()` stops asking. Both are synchronous, need no tap, can be called at any time and may be repeated. On a static widget they do nothing (its shared state, like every chat state, is pushed anyway), and a preview never watches.

The refresh period is a property of the shared widget: 30 seconds when created, changed only with `cache_time` in editSharedWidgetData (5-3600 seconds: less than 5 is refused, more than 3600 is clamped). Choose it for how often the shared state changes: about 10 seconds for a price, minutes or an hour for something that changes slowly.

Keep the newest state. A poll can bring a shared state older than one the widget already drew from a request's answer or an earlier poll, and counts would jump back and forth. Put a revision in the shared state (`"r": 41`, raised on every edit, and in the answer too) and have the widget ignore anything older than the newest it has seen.

Widgets in groups

Bots can send widgets to basic groups and supergroups they are members of, forum topics included (test server only). The bot needs the right to send other messages (`can_send_other_messages` in ChatPermissions: animations, games, stickers and inline bots). Widgets are checked like messages sent via an inline bot, so a group that does not give the bot this right refuses the widget too. Channels refuse widgets, sent directly or via the bot.

In a group:

- **The chat state is shared by all members.** Everyone sees the same chat state. Draw it per user with `TelegramWidget.viewer.id` (chess for two is one widget and one chat state).
- **Requests run on a tap only**, never in the background.
- **The bot addresses the message as usual**: the `chat` and `message_id` of a widget_query are the ones it uses for its other messages in that group, and editMessageWidgetData takes

the same pair.

Inline mode widgets

A bot with inline mode enabled can return widgets as inline results (test server only): an InlineQueryResultArticle whose `input_message_content` is a widget, in answerInlineQuery. A widget result cannot be sent in a channel. The content has the same two forms as sending:

- InputWidgetMessageContent: a new widget, with the fields of `sendWidget` (`html`, `width`, `height`, `public_data`, `chat_data`, the caption, `allow_queries`, `is_shared`). The server checks the code and the state when it saves the results (same limits, same errors). **Each send** of a result with `is_shared` creates **its own new shared widget**.
- InputSharedWidgetMessageContent: an existing shared widget, by `widget_id`, with this message's `chat_data` and caption. Every send shows that shared widget.

```
POST https://api.telegram.org/bot<token>/answerInlineQuery
{
  "inline_query_id": "<inline_query_id>",
  "results": [{
    "type": "article",
    "id": "ton",
    "title": "TON price",
    "description": "A live quote",
    "input_message_content": { "widget_id": "7770011223344556677", "caption":
"TON price" }
  }]
}
```

- **The list of results shows the widget itself**, running as a preview with the result's state (for a shared widget, its code and shared state): it takes no taps (a tap selects the result) and knows nothing about the user.
- **Every send becomes its own message** via the bot, with its own chat state.
- **Requests from a message sent via the bot run on a tap only.** The bot gets them as a widget_query with `inline_message_id` instead of `chat` and `message_id`, together with the `chat_instance` and `chat_type` of the chat where the user tapped, and changes the message's state with editMessageWidgetData and that `inline_message_id`.
- Only apps that support widgets receive widget results.

> **Note:** A bot does not receive the messages users send via it. To show one live shared state through inline mode, create the shared widget once with `sendWidget` and offer results by its identifier. A result with `html` and `is_shared` makes a new shared widget per send, and the bot sees its identifier only in a later request.

Requests to the bot

A request is a callback button inside the widget. The widget calls `TelegramWidget.request(data)` with 1-64 bytes of data. The bot receives a widget_query update (or a shared_widget_query from a forward) with who tapped, the request's data and the context. It answers with answerWidgetQuery and optional `data` of up to 1024 bytes, which only the widget that asked receives, as its `result`.

In the widget:

```
button.addEventListener('click', () => {
  TelegramWidget.request('vote:2').then(
    (answer) => show(answer.result),   // { result: string | null }
    (error) => showError(error.code),  // error.name === 'WidgetRequestError'
  );
});
```

- **One tap, one call.** A request on a tap must be made in the tap's handler, synchronously, on the visible widget, and not from a timer, a promise callback or a later frame. One tap unlocks one call, shared with `openLink`, `openTelegramLink` and `copyText`.
- **One at a time.** A second call while a request waits for its answer rejects with `BUSY`.
- The promise resolves with `{ result }`: the bot's `data`, or `null` if it answered without any. It rejects with a WidgetRequestError whose `code` says why: `NOT_ALLOWED`, `BUSY`, `RATE_LIMITED`, `TIMEOUT`, `FAILED`, `INVALID_ANSWER`.

Where a widget may ask is decided by the server for each message, from the widget's flags (a static widget's message, or the shared widget), and passed to the widget by the app:

Message	On a tap	In the background	`viewer`	`chatType`
the original in the private chat with the bot	Yes	Yes	the user	`'private'`
an original in a group	Yes	No	the user	`'group'`, `'supergroup'`
an original sent via the bot	Yes	No	the user	the chat's type
a forward of a shared widget	Yes	No	the user	`null`
a forward of a static widget	No	No	`null`	`null`

This is for a widget sent with `allow_queries`. A widget without it asks nowhere, its originals included, and has no `viewer`. Elsewhere `request()` rejects with `NOT_ALLOWED`.

Background requests. Only the original in the private chat with the bot may ask without a tap, and only while it is visible: one request when the widget appears on screen (scrolled into view,

the app back in front), then at most one every 5 seconds while it stays visible. Apps create a widget before they show it, so a request the script makes in its first lines waits and goes out as soon as the widget appears. Ask for fresh state right at the start. Use background requests to refresh what this user sees while the chat is open. For what is the same for everyone, a shared widget's shared state needs no requests at all. Neither the server nor the bot can tell a background request from a tap.

In the bot. A request from an original arrives in the field `widget_query` of Update, and one from a forwarded copy of a shared widget in the field `shared_widget_query`. If you pass `allowed_updates`, include both. Their fields depend on where the user tapped:

Field	Original (private chat or group)	Original sent via the bot	Forward of a shared widget
update	`widget_query`	`widget_query`	`shared_widget_query`
`id`, `from`, `data`	Yes	Yes	Yes
`chat`, `message_id`	Yes	No	No
`inline_message_id`	No	Yes	No
`chat_instance`	Yes	Yes, the chat of the tap	No
`chat_type`	No, see `chat.type`	Yes, the chat of the tap	No
`widget_id`	for a shared widget	for a shared widget	always

```
{
  "update_id": 100000001,
  "widget_query": {
    "id": "8123456789012345678",
    "from": { "id": 111111111, "is_bot": false, "first_name": "Alice",
"language_code": "en" },
    "chat": { "id": 111111111, "type": "private", "first_name": "Alice" },
    "message_id": 4242,
    "chat_instance": "901234567890123456",
    "widget_id": "7770011223344556677",
    "data": "refresh"
  }
}
```

The query does not carry the widget's state: keep the authoritative state on your side, keyed by the message (or by `widget_id`), and check every request against it.

Answering. Call answerWidgetQuery for **every** query, with or without `data`, within 15 seconds. Otherwise the widget's promise rejects with `TIMEOUT`. `data` is the only thing an answer carries:

no change of state, no notification, no alert, no URL. The app shows nothing for an answer, and the widget shows the result itself.

```
POST https://api.telegram.org/bot<token>/answerWidgetQuery
{ "widget_query_id": "8123456789012345678", "data": "{\"p\":5.47}" }
```

- **The answer does not change the state.** To change the chat state or the shared state, call the methods that change it, before or after answering. The widget gets `chatDataChanged` / `publicDataChanged` when the new state arrives, which can be before or after the answer. A widget that needs the new state at once (the user's own vote, the new board) gets it from the bot in the answer.
- **Answer also when something failed**, with a short text the widget can show: an unanswered request ends in the widget as `TIMEOUT`.
- **`from` and `widget_id` cannot be forged.** `from` is the user who tapped, authenticated by Telegram: no signature to check. `widget_id` is computed by the server from the tapped message, so a request cannot claim to come from another of your shared widgets. A forward's request carries nothing else about its place, so key “one vote per user” and paid actions on the pair **(`widget_id`, `from.id`)**.
- **Treat `data` as untrusted input**, like `callback_data`: check it against the state you keep.
- Do slow work after answering, then change the state.
- Requests are limited to 5 per 5 seconds per user and bot, background requests included.

Cause	The widget gets
the widget was sent without `allow_queries`, a forward of a static widget, no tap where background requests are off, or a preview	`NOT_ALLOWED`, before anything is sent
`data` empty or over 64 bytes	`TypeError`, before anything is sent
the bot did not answer in 15 seconds	`TIMEOUT`
over 5 requests per 5 seconds per user and bot	`RATE_LIMITED`
an answer the app cannot accept (it never reaches the widget's code)	`INVALID_ANSWER`
the message is gone, the bot is blocked or deleted, the network failed	`FAILED`

Changing the state

Only the bot that sent the widget changes its state. Every edit is **silent**: no “edited” mark, no notification, the chat does not move up in the chat list.

Message	Chat state	Shared state of a static widget	Shared state of a shared widget
original (private chat, group)	editMessageWidgetData with `chat_id` + `message_id`	same	editSharedWidgetData by `widget_id`

Message	Chat state	Shared state of a static widget	Shared state of a shared widget
original sent via the bot	editMessageWidgetData with `inline_message_id`	same	editSharedWidgetData by `widget_id`
forward	none	none: a copy	editSharedWidgetData by `widget_id`

```
POST https://api.telegram.org/bot<token>/editMessageWidgetData
{ "chat_id": -1001234567890, "message_id": 77, "chat_data": "{\"b\":\"x---o----
\",\"t\":\"x\"}" }

POST https://api.telegram.org/bot<token>/editSharedWidgetData
{ "widget_id": "7770011223344556677", "public_data": "{\"p\":5.47}",
"cache_time": 10 }
```

- **An edit that changes nothing** fails with `Bad Request: message is not modified`, as `editMessageText` does: take it as done (and still answer the request that led to it).
- **Commit after the edit.** Put a user's move into your own state only once the edit went through, so a failed edit does not take a turn away.
- **Rate limits.** An `editMessageWidgetData` call counts as one sent message against your bot's limits for sending messages. A shared widget takes at most one edit per second. Above them the method fails with a flood wait. When many users act at once (votes), coalesce your edits: keep the state on your side and write it at most once a second.
- **A deleted original's** chat state can no longer be changed, but a shared widget lives on: change its shared state by `widget_id`.
- `editSharedWidgetData` is the only way to set a shared widget's refresh period (`cache_time`).

Forwards

Users can forward a widget to any chat, groups and channels included (test server only). The forward runs the same code in the same sandbox:

Property	Original	Forward
Code, size anchors, shared state of a static widget	as sent	copies as of the moment of forwarding, never updated
Code, size anchors, shared state of a shared widget	the shared widget's	the same shared widget's
Chat state	the current one	**none**, ever
`TelegramWidget.isForward`	`false`	`true`
`TelegramWidget.chatType`	the chat's type	`null`: a forward is told nothing about the chat it is in

Property	Original	Forward
Requests	per the table above	on a tap, only for a shared widget sent with `allow_queries`, never in the background
`TelegramWidget.viewer`	where it may ask	only if it may ask
Web Storage	this message's	its own, starts empty

- **A widget is never forwarded without its author.** The forward header stays even when the user chooses to hide the sender, so a widget always shows which bot it comes from.
- **A request from a forward tells the bot nothing about where it is**: it arrives as a shared widget query with only `id`, `from`, `data` and `widget_id`: no chat, no message, no `chat_instance`. The bot answers by the query's id and changes the shared state by `widget_id`, even if the original was deleted.
- **Live forwards need a shared widget sent with `allow_queries`.** A forward of a static widget shows the shared state as it was and cannot ask. Point its readers to the bot with `openTelegramLink('https://t.me/<bot_username>?start=…')`.
- **Widgets cannot be copied**: copyMessage and copyMessages refuse them. A widget sent with `protect_content` cannot be forwarded.

Previews

In the list of inline results, before the user sends one, the app shows a widget result as a **preview**: `TelegramWidget.isPreview` is `true`. A preview runs with the result's state (for a shared widget, its code and shared state), with timers, animations and the theme as usual, but it is not a message yet and not the user's:

- It gets no taps or pointer events (a tap selects the result).
- `request()` rejects with `NOT_ALLOWED`. `openLink`, `openTelegramLink`, `copyText` and haptics do nothing (`ready` and `setResizeAnchor` work), and `watchPublicData()` has no effect.
- `chatType`, `viewer` and `wallet` are `null`, and `isForward` is `false`.
- Web Storage starts empty and nothing is kept.

Draw the first state a sent widget would draw: the preview is what the user is about to send.

Privacy

What the bot learns, only from requests:

- Who tapped (`from`, a full User, as in a callback query) and the `data` the widget sent.
- For an original: the chat and the message (or `inline_message_id`), `chat_instance` and, via the bot, the chat type.
- For a forward: only the shared widget's `widget_id`.
- Whatever the widget's code puts into the 64 bytes of `data`: its language, theme, text size, bubble width, time zone, what it keeps in Web Storage.
- In the private chat, background requests show when the user has the chat open with the widget on screen.

What the bot never learns: anything about a user who does not tap a widget (outside the private chat's background requests), where a widget was forwarded (no chat, chat type or chat instance), whether a link was opened, or anything about a preview.

What the widget knows: its state, the theme, the language, and whether it is a forward or a preview. It knows `chatType` in originals only, and the `viewer` (id, first and last name, username, Premium flag) only where it may send requests. These are the same fields the bot gets as `from` with a tap. A widget that may not send requests does not know who is looking and cannot pass that out through a link either. In its private chat, a bot that wants the widget to know the user can put the id into the chat state itself.

Storage, links and haptics

Web Storage. `localStorage` is kept for this message on this device. It is in place from the first line of script, survives the widget being destroyed and re-created and app restarts, and is deleted with the message, the chat, the account and on “Clear cache”. It is never sent to the bot or the server and is not synced between devices: what must follow the user belongs in the chat state, which the bot writes. Each area holds 4 KB (keys and values in UTF-8). A `setItem` beyond that throws `QuotaExceededError` and stores nothing, so catch it. Keep this user's own choices there (a best score, “voted”). `sessionStorage` follows the same rules and lives until the app restarts. A forward, or another message showing the same shared widget, is another message, and its storage starts empty.

Links. `openLink(url)` opens an `https:` URL of at most 256 characters (no user name or password, not a `t.me`, `telegram.me` or `telegram.dog` host) after the user confirms it, shown in full. `openTelegramLink(url)` opens a `https://t.me/…` link inside Telegram without a confirmation of its own, as a tapped `t.me` link in a message does (Telegram's own handling still confirms what it always does, such as joining a chat or a proxy). Both work only in the handler of a real tap, once per tap. A tapped `<a href="https://…">` and `window.open(url)` do the same, through `openTelegramLink` for Telegram hosts and `openLink` otherwise. A URL these methods do not accept (not `https:`, too long, with a user name or password, a host of the other method, `tg:` links) throws `TypeError`. A tapped link or `window.open` with another scheme (`http:`, `javascript:`) is ignored. The widget is not told whether the user opened the link.

Haptics. `HapticFeedback.impactOccurred`, `notificationOccurred` and `selectionChanged` work as in Mini Apps, only while the widget is visible, at most 5 in a burst and 10 per second. They are free while a finger is down on the widget and for 5 seconds after it is lifted. Without a touch, the first call opens a one-second series and later ones are dropped. The next series may start after a touch or 10 seconds later. So a timer cannot keep buzzing the phone, while a tap, and a bot's answer that arrives a few seconds after it, can play feedback. Desktop and web apps play nothing.

copyText. `TelegramWidget.copyText(text)` copies 1-256 characters to the clipboard on a tap, like an inline button's `copy_text`. The app shows a toast with the copied text, so a widget cannot show one text and copy another unnoticed. `navigator.clipboard` is not available.

Payments with the wallet

A widget can ask the user to pay or to sign through Telegram's built-in wallet. The widget is the front of a dApp and the bot is its back end, as a site and its server are with TON Connect. The formats are TON Connect's, so a bot reuses its TON Connect code, but there is no connection, manifest or bridge: who asks is the bot that sent the message, and the app's own wallet window, outside the bubble, says so.

```
// A tap asks the bot with the user's wallet; the bot answers with a fresh
transaction; the widget passes it on.
TelegramWidget.request('pay:42', { wallet: true })
  .then((a) => TelegramWidget.wallet.sendTransaction(JSON.parse(a.result),
'paid:42'))
  .then((a) => show(a.result), (e) => showError(e.code));
```

1. **Check `TelegramWidget.wallet`.** It is `null` where the user has no wallet in this app (an older app, a country where the wallet is unavailable), where the widget may not send requests, and in a preview. Draw a pay button only when it is there.
2. **On a tap, ask the bot with the wallet:** `request(data, { wallet: true })`. The bot's query has `type` `{ "type": "data", "wallet_card": … }`: the TonWalletCard of the user's profile, its address and balance. A user without a wallet gets one created then. Never in the background.
3. **The bot answers with a fresh transaction**, TON Connect's `sendTransaction` request: `{ valid_until, messages: [{ address, amount, payload?, stateInit? }] }`, 1-255 messages, each `address` user-friendly, each `amount` in nanotons as a decimal string. Leave out `from`: the wallet that pays is the one on the user's device, which can differ from the profile's card (with the wallet's backup off, a device keeps its own key and wallet).
4. **In the answer's handler, the widget calls `wallet.sendTransaction(transaction, data)`.** The answer to a call made on a tap allows one wallet call within 10 seconds, so no second tap is needed. The app's window shows the bot, the amounts and the destinations, and the user confirms.
5. **The bot gets the outcome** as a query whose `type` is `{ "type": "sent_transaction", "wallet": …, "msg_hash": … }`, with the call's `data`: the hash of the sent message and the device's wallet that sent it (WidgetQueryWallet). It checks the transaction on the chain (the sender, the recipients, the amounts, the outcome), not by the hash alone, and answers. The widget's promise resolves with that answer.

```
{
  "update_id": 100000004,
  "widget_query": {
    "id": "8123456789012340002",
    "from": { "id": 111111111, "is_bot": false, "first_name": "Alice" },
    "chat": { "id": 111111111, "type": "private", "first_name": "Alice" },
    "message_id": 4243,
    "chat_instance": "901234567890123456",
    "type": {
```

```
      "type": "sent_transaction",
      "wallet": { "address": "UQ…", "amount": "1250000000", "public_key":
"a3f1…" },
      "msg_hash": "…"
    },
    "data": "paid:42"
  }
}
```

`wallet.signData(payload, data)` asks for a signature the same way: `payload` is TON Connect's `{ type: 'text', text }`, `{ type: 'binary', bytes }` or `{ type: 'cell', schema, cell }`. The bot gets a query whose `type` is `{ "type": "sign_data", "wallet": …, "sign_data": … }`: TON Connect's `signData` result as JSON, with the wallet that signed. The domain signed for is `bot<bot_id>`. Check the signature with the wallet's `public_key`, and tie the key to the wallet's address on the chain.

One wallet call per tap or per answer, and one call in flight together with requests. Besides the request codes, the promise rejects with `REJECTED` (the user declined) or `BAD_REQUEST` (the wallet refused the transaction or payload). A malformed transaction throws `TypeError`. After `TIMEOUT` the payment may have gone out, so ask the bot about the order and never pay again blindly.

Lifecycle

- **Off screen** (scrolled away, the app in the background) a widget is paused: `document.hidden` is `true`, `visibilitychange` fires, `TelegramWidget.isActive` turns `false` with the `deactivated` event, animation frames stop and timers fire at most once per second. It resumes with `activated`.
- **A widget can be destroyed and re-created at any time** (scrolled far away, memory pressure, app restart). Only the widget's state and Web Storage survive: redraw from them when the script starts.
- `TelegramWidget.ready()` tells the app the first state is drawn. It is optional.
- A widget that goes past one of its limits (CPU, memory, nodes, text, operations, canvas, paint) or crashes is stopped: its last frame stays with a “Widget stopped” badge. The app rebuilds it when it is shown again. After two crashes in a row it stays stopped until the app restarts.

What widgets support

The **widget profile** is the contract between widgets and Telegram apps: a feature is in when every app draws it the same way, and every app removes what is out before drawing. Something out of the profile never fails the widget: the declaration, selector, element or event is removed and the rest works. This page describes **profile 1**. A later profile only adds features.

HTML

In:

- Text and structure: `div`, `span`, `p`, `br`, `hr`, headings `h1` – `h6`, `header`, `footer`, `main`, `nav`, `section`, `article`, `aside`, `pre`, `blockquote`, `figure`, `figcaption`.
- Inline text elements: `b`, `i`, `u`, `s`, `em`, `strong`, `small`, `mark`, `code`, `kbd`, `sub`, `sup`, `abbr`, `cite`, `q`, `del`, `ins`, `time`, ….
- Lists (`ul`, `ol`, `li`, `dl`, `dt`, `dd`, `menu`), tables with `colspan` and `rowspan`, `details` and `summary`.
- `<img>` with a `data:` PNG, JPEG, GIF or WebP, `<canvas>`, `<button>`.
- Links: `<a href="#id">` (no navigation) and `<a href="https://…">` (opens through `openLink` on a tap).
- Attributes: `class`, `id`, `style`, `title`, `hidden`, `lang`, `dir`, `data-*`, `aria-*`, and inline `on*` handlers (they run in the sandbox).

Unknown elements render as inline boxes.

Out:

- Form controls: `input`, `select`, `textarea`, `option`, `fieldset`, … are removed with their content. `<form>` becomes a block and `<label>` an inline box.
- `progress`, `meter`, `dialog` and popovers, `audio`, `video`, `iframe`, `object`, `embed`, external `<script src>`, `link`, `meta`, `template`, `math`, `ruby`.
- Images from anything but a `data:` URL (shown as a placeholder, never requested), and `data:` SVG, AVIF, BMP or ICO images.
- Legacy presentational elements and attributes (`center`, `font`, `align`, `bgcolor`, …).
- `contenteditable`, `draggable`, `tabindex`, `autofocus`, `accesskey`.

Inline SVG

In: shapes and paths, `g`, `defs`, `symbol`, `use` (fragment links), `transform`, `viewBox`, `text` and `tspan`, linear and radial gradients, CSS on SVG (fills and strokes from stylesheets, CSS animation of SVG properties).

Out: `clipPath`, `mask`, `pattern`, `marker`, filter primitives, SMIL animation (`animate`, `animateTransform`, `animateMotion`), `foreignObject`, `<image>`.

CSS

In:

- Layout: block, inline, inline-block, flexbox (with `gap`), grid, tables, `position: relative / absolute / fixed` (fixed is relative to the bubble), `z-index`, `float`, `aspect-ratio`, `box-sizing`, `object-fit`, `overflow: hidden / clip`.
- Box and paint: margins, padding, borders (solid, dashed, dotted, double), `border-radius`, `outline`, `box-shadow`, `opacity`, `visibility`, backgrounds with `data:` images, linear, radial and conic gradients, 2D transforms (and the `translate`, `rotate`, `scale` properties).

- Filters, if every app draws them: `filter` with `blur()`, `brightness()`, `drop-shadow()`, and `backdrop-filter` with `blur()`, `brightness()`.
- Text: `font-size`, `font-weight`, `font-style`, `line-height`, the `font` shorthand, generic families only (`system-ui`, `sans-serif`, `serif`, `monospace`, `ui-sans-serif`, `ui-serif`, `ui-monospace`, `ui-rounded`, with `-apple-system` turned into `system-ui` and other names removed), `font-variant-numeric: tabular-nums`, `text-align`, `text-decoration`, `text-transform`, `letter-spacing`, `white-space`, `overflow-wrap`, `word-break`, `text-overflow: ellipsis`, `vertical-align`, list styles, right-to-left text, color emoji, `user-select: none / text / auto`.
- Colors: named, hex, `rgb()`, `hsl()`, `transparent`, `currentColor`, `light-dark()` and `color-scheme` (resolved to the chat theme).
- Motion: transitions, `@keyframes` animations, timing functions (`cubic-bezier()`, `steps()`, `linear()`), `animation-play-state`, the animation and transition events.
- Values: `px`, `em`, `rem`, `%`, `ch`, `ex`, `vw` / `vmin` / `vmax` (the bubble's width), `calc()`, `min()`, `max()`, `clamp()`, custom properties and `var()`, `env(safe-area-inset-*)` (the time in the corner), `content` with strings, `attr(data-*)` and counters.
- Selectors: type, class, id, attribute, combinators, structural pseudo-classes (`:first-child`, `:nth-child()`, `:not()`, …), `:is()`, `:where()`, `:root`, `::before`, `::after`, `:active`, `:hover` (with a mouse only).
- At-rules: `@supports`, `@keyframes`, and `@media` on the width, the orientation, `prefers-color-scheme` and `prefers-reduced-motion`.
- `touch-action: auto / pan-y / none`, `pointer-events`, `cursor`.

Out:

- `vh` and other height-relative units.
- `overflow: auto / scroll` (becomes `hidden`) and `position: sticky` (becomes `relative`).
- Logical properties (`margin-inline`, …), multi-column layout, `display: contents / flow-root / list-item`.
- 3D transforms and `perspective`, other filter functions (`grayscale()`, `saturate()`, …), `clip-path`, masks, blend modes.
- `text-shadow`, `-webkit-line-clamp`, `background-clip: text`, `writing-mode`.
- Modern color functions (`oklch()`, `color-mix()`, …).
- `env()` names other than the safe-area insets, and `url()` with anything but a `data:` image or a `#fragment`.
- `@font-face` and named fonts, `@import`, `@container`, `@layer`, `@property`, CSS nesting.
- `:has()`, `:focus*`, `::first-letter`, `::first-line`, `::marker`, `:target`, and form-state selectors (`:checked`, …). `:disabled` and `:enabled` work for `<button>`.
- `@media` on hover, pointer, resolution or print.

JavaScript

In:

- ES2023 (QuickJS), Promises and `async` / `await`, timers, `requestAnimationFrame`.
- The DOM: `createElement`, `querySelector`, `innerHTML` parsed in the sandbox, `classList`, `dataset`, `style`, `addEventListener`, `CustomEvent`, `MutationObserver`.

- Layout reads answered by the app: `getBoundingClientRect`, `offsetWidth`, `getComputedStyle`, `matchMedia`, `elementFromPoint`, `ResizeObserver`. `IntersectionObserver` always reports visible.
- Canvas 2D: paths, `Path2D`, gradients, text and `measureText`, `drawImage` from `data:` images and canvases, transforms, clipping, `isPointInPath`.
- Web Animations (`element.animate`, `getAnimations`), if every app draws them.
- `Intl` in the user's language (`navigator.language`).
- `crypto.getRandomValues`, `crypto.randomUUID`, `TextEncoder`, `TextDecoder`, `URL`, `URLSearchParams`, `structuredClone`, `AbortController`, `atob`, `btoa`.
- `localStorage` and `sessionStorage`, `document.hidden` and `visibilitychange`.
- `eval` and `new Function` (inside the sandbox).

Out:

- Every network API: `fetch`, `XMLHttpRequest`, `WebSocket`, `EventSource`, `RTCPeerConnection`, `sendBeacon`, `importScripts`, `Worker` and `BroadcastChannel` throw `NetworkDisabledError`.
- Navigation: `location`, `history`. `window.open` goes through `openLink`.
- `alert`, `confirm`, `prompt`, `print`. No dialog is shown, `confirm` returns `false` and `prompt` returns `null`.
- `navigator.clipboard`, `share`, `vibrate`, geolocation, media devices, notifications.
- `attachShadow`, custom elements, `Blob`, `FileReader`, `URL.createObjectURL`, `requestFullscreen`.
- `focus()` and `blur()`, which do nothing.
- In canvas: `getImageData` / `putImageData`, `toDataURL`, `createPattern`, `createConicGradient`, shadows, `globalCompositeOperation`, `ctx.filter`.
- WebGL and `OffscreenCanvas`.

Libraries are not bundled: a `<script src>` naming a CDN library is an external script like any other and does not run.

Input

In:

- `click`.
- Pointer events (`pointerdown`, `pointermove`, `pointerup`, `pointercancel`, enter, leave, over, out) with coordinates, `pointerType` and `setPointerCapture`, and the mouse compatibility events.
- `dblclick` (always the widget's) and `contextmenu` on a hold over `touch-action: none`.
- `preventDefault` on `pointerdown`.
- Animation and transition events, `toggle` of `<details>`.

Who gets each press is described in Gestures.

Out:

- Touch events (use pointer events).
- Keyboard events and keyboard focus. There is no text input of any kind, and a key press never counts as a tap.

- Wheel and scroll events.

Detecting features

- `TelegramWidget.profile` is the profile the app enforces (`1`), and `TelegramWidget.isProfileAtLeast(n)` returns `profile >= n` .
- `CSS.supports(property, value)` , `CSS.supports(condition)` and `@supports` answer from the profile: `true` only when the app keeps that declaration.
- Methods added within a profile are detected with `typeof` , for example `typeof TelegramWidget.setResizeAnchor === 'function'` .
- Outside Telegram (a plain browser) `window.TelegramWidget` is `undefined` .

Limits

Limit	Value	Beyond it
HTML	32768 bytes of UTF-8	`INPUT_MEDIA_WIDGET_TOO_LARGE`
Compressed HTML	4096 bytes after zlib at level 9 (Counting the size)	`INPUT_MEDIA_WIDGET_TOO_LARGE`
HTML text	valid UTF-8 without the NUL character (U+0000)	`INPUT_MEDIA_WIDGET_HTML_INVALID`
Chat state, shared state (`chat_data` , `public_data`)	a text of up to 1024 bytes each (bytes of UTF-8, not characters)	`INPUT_MEDIA_WIDGET_DATA_TOO_LARGE` , at sending and in edits
Chat state, shared state text	valid UTF-8 without the NUL character (U+0000)	`INPUT_MEDIA_WIDGET_DATA_INVALID` at sending, `WIDGET_DATA_INVALID` in edits
Request `data`	1-64 bytes of UTF-8	`TypeError` in the widget
Answer `data`	1024 bytes of UTF-8	`WIDGET_RESULT_TOO_LARGE`
Time to answer	15 seconds, which is also how long a query's id is valid	`TIMEOUT` in the widget, and a late answer is refused (`query is too old`)
Requests per user and bot	5 per 5 seconds	`RATE_LIMITED` in the widget

Limit	Value	Beyond it
Requests per widget	one in flight, one per tap (shared with links and `copyText`)	`BUSY` or `NOT_ALLOWED`
Background requests	private chat with the bot only, while visible: one when it appears, then one per 5 seconds	`NOT_ALLOWED` or `RATE_LIMITED`
`editMessageWidgetData`	counts as one sent message against the bot's sending limits	flood wait
Edits of a shared widget	1 per second	flood wait
Refresh period of a shared widget	30 seconds when created. `cache_time` sets it to 5-3600 seconds (more is clamped). Apps poll only while a widget that called `watchPublicData()` is on screen	`invalid cache time specified` below 5
Bubble	200 px to the bubble's maximum wide, 50-600 px tall	clipped
`localStorage`, `sessionStorage`	4 KB per area per message (keys and values in UTF-8)	`QuotaExceededError`
`copyText`	1-256 UTF-16 code units, on a tap	`TypeError`, or dropped without a tap
URLs for `openLink`, `openTelegramLink`	256 characters	`TypeError`
Handlers per event type	16	further handlers are ignored
Haptics	visible widgets only, 5 in a burst, refilled at 10 per second. Free during a touch and 5 seconds after it, otherwise one 1-second series, the next one after a touch or 10 seconds later	dropped
CPU	300 ms of JavaScript per second	timers and frames wait, and a task that runs past the quota makes the next ones wait longer
CPU in one task	500 ms of continuous JavaScript, the first run of the script included	the widget is stopped
Layout reads	128 blocking queries (`getBoundingClientRect`, `offsetWidth`, `getComputedStyle`, …) in one task, taking at most 1000 ms together	the widget is stopped
The app's main thread	the widget's layout reads plus about 25 µs per DOM change and 2 µs per canvas command: 100 ms per second	timers and frames wait
Memory	8 MB	the widget is stopped

Limit	Value	Beyond it
Timers	64 live timers	`RangeError`
Nodes	2000 in the document: elements, text, comments	the widget is stopped
DOM changes	10000 in one task: changes and element method calls such as `scrollTo`	the widget is stopped
Text	32 K UTF-16 units per text value and 48 K in the document (text, visible, `data-*` and `aria-*` attributes, CSS strings)	the widget is stopped
Complex-script text	8192 units	the text is refused
Attributes	8 K UTF-16 units per attribute or inline style	the widget is stopped
CSS	400 rules and 32 `@keyframes`	the widget is stopped
Canvas commands	4000 per canvas in one task	`RangeError`
Canvas pixels	2048 × 2048 for all the widget's canvases together (a canvas is 300 × 150 until sized)	the widget is stopped
`data:` images	1024 × 1024 pixels	a placeholder
Script animations	64 running (`element.animate`)	further ones are not drawn
Heavy elements	16 at once with a blurred `box-shadow` , a `filter` , a `backdrop-filter` or `opacity` below 1, or a running animation of one of them (a `::before` / `::after` counts as an element), at most 2 of them with `backdrop-filter` . A shadow without blur, an element under `display: none` and the dimming of a disabled or pressed `<button>` do not count	the widget is stopped, and its last frame is drawn without these effects
Crashes	2 in a row	the widget stays stopped until the app restarts

The counted limits (nodes, DOM changes, text, attributes, CSS, canvas, heavy elements) are the same in every app and stop the same widget on a slow phone and on a fast computer. The time quotas only slow a heavy widget down. Of the time limits, only one task running past 500 ms, or layout reads past 1000 ms in one task, stops the widget. Keep well inside the limits and measure on an older phone.

Initializing widgets

Widgets need no script tag. Inside Telegram, `window.TelegramWidget` exists before the widget's first script runs, with the state, the theme and the context already in place. Outside Telegram it is

`undefined`: check for it if you open a widget in a browser.

TelegramWidget

Field	Type	Description
profile	Integer	The widget profile the app enforces: `1` in this version.
isProfileAtLeast(n)	Function	Returns *true* if `profile` is at least `n`, and *false* for anything but a number.
colorScheme	String	The chat's color scheme: `light` or `dark`. Also the `--tg-color-scheme` variable.
themeParams	ThemeParams	The bubble's palette, a new frozen object after every change.
isActive	Boolean	*True*, if the widget is visible (`!document.hidden`).
isPreview	Boolean	*True*, if this is an inline result shown in the list of results before it is sent. See Previews.
isForward	Boolean	*True*, if this is a forwarded copy.
chatType	String	The kind of chat the message is in: `private`, `group`, `supergroup` or `channel`. `null` in a forward, in a preview, and when the app does not say.
viewer	WidgetViewer	The user looking at the widget, only where the widget may send requests, otherwise `null`. Frozen.
publicData	String	The widget's shared state: a text of up to 1024 bytes, or `null`. Seen wherever the widget is shown: a forward has a static widget's copy as of the forward, or a shared widget's current shared state. Read-only.
chatData	String	The widget's chat state: a text of up to 1024 bytes, or `null`. Tied to this message, and in a group one for all members. Always `null` in a forward. Read-only.
watchPublicData()	Function	Asks for the live shared state: while this widget is on screen, the app polls its shared widget's shared state at the server's refresh period (one poll per `widget_id` for all widgets showing it) and fires `publicDataChanged` on a new shared state. Without it, a shared widget's shared state arrives only with the message. Synchronous, needs no tap, may be called at any time and repeated. Does nothing for a static widget, whose state is pushed anyway, and in a preview.
unwatchPublicData()	Function	Stops asking for the live shared state (see `watchPublicData`). Synchronous, may be repeated.

Field	Type	Description
request(data[, options])	Function	Sends a request to the bot with `data`, a string of 1-64 bytes in UTF-8 (anything else throws `TypeError`). Returns a Promise that resolves with a WidgetRequestAnswer or rejects with a WidgetRequestError. Only in the handler of a tap, or in the background where the message allows it. One request at a time. `options` may be `{ wallet: true }`, on a tap only: the bot gets the user's wallet with the request (Payments with the wallet). See Requests to the bot.
copyText(text)	Function	Copies `text`, 1-256 UTF-16 code units, to the clipboard, and the app shows a toast with it. Only in the handler of a tap, once per tap.
openLink(url)	Function	Opens an external `https:` link (at most 256 characters, not a `t.me`, `telegram.me` or `telegram.dog` host) after the user confirms it, shown in full. Only in the handler of a tap, once per tap. An invalid URL throws `TypeError`.
openTelegramLink(url)	Function	Opens a `https://t.me/…` link (at most 256 characters) inside Telegram, without a confirmation of its own, as a tapped link in a message. Same rules as `openLink`.
HapticFeedback	HapticFeedback	Haptic feedback.
contentSafeAreaInset	ContentSafeAreaInset	The part of the widget the app's time covers, in CSS pixels. A new frozen object after every change, with the `contentSafeAreaChanged` event. See The time in the corner.
wallet	WidgetWallet	Telegram's wallet, or `null` where the user has no wallet in this app, where the widget may not send requests, and in a preview. See Payments with the wallet.
setResizeAnchor(edge)	Function	Which edge of the bubble stays in place when the widget's height changes: `top` (the default) or `bottom`. Applies to the changes that follow and needs no tap. Anything else throws `TypeError`. The earliest apps with widgets lack it, so check `typeof TelegramWidget.setResizeAnchor === 'function'`.
ready()	Function	Tells the app the first state is drawn. Optional: the app also notices the first content itself.
onEvent(eventType, eventHandler)	Function	Sets the handler for an event. The handler is called with `this` bound to `TelegramWidget` and no arguments. At most 16 handlers per event type. Unknown types are ignored.
offEvent(eventType, eventHandler)	Function	Removes a handler set with `onEvent`.

All methods are synchronous and return nothing, except `isProfileAtLeast`, `request` and the wallet's methods (a Promise) and the haptic methods (they return `HapticFeedback` for chaining). The deprecated aliases `theme` (= `colorScheme`) and `onTheme(callback)` still work. `sendMessage` does not exist: nothing is ever sent to the chat on the user's behalf. The widget is not told its `widget_id`.

ThemeParams

The colors of the bubble the widget sits in, as `#rrggbb` strings (lowercase), with the keys of Mini Apps' `themeParams`. Every key is always present.

Field	Type	Description
bg_color	String	The bubble's color, behind the widget: an incoming bubble's, or the user's outgoing bubble's for a widget they sent via the bot or forwarded.
text_color	String	Message text.
hint_color	String	Secondary text: time, meta.
link_color	String	Links.
button_color	String	The app's primary button.
button_text_color	String	Text on the primary button.
secondary_bg_color	String	A second surface on the bubble (quote, inline button background).
accent_text_color	String	Accent text.
subtitle_text_color	String	Subtitles.
destructive_text_color	String	Destructive actions.
section_bg_color, section_header_text_color, section_separator_color, header_bg_color, bottom_bar_bg_color	String	No bubble counterpart. Filled with the closest color so that Mini Apps code works. `section_separator_color` draws table lines and `<hr>`.

CSS variables

The app sets these on the widget's root element, inline and `!important`: a widget cannot override them on the root, only redefine them on its own elements.

Variable	Description
`--tg-theme-<key>`	Every ThemeParams key with `_` turned into `-`: `--tg-theme-bg-color`, `--tg-theme-text-color`, `--tg-theme-hint-color`, ...

Variable	Description
`--tg-color-scheme`	`light` or `dark`.
`--tg-text-font-size`	The chat's message text size, which follows the user's text size setting (about `15px`).
`--tg-text-line-height`	The message text's line height, unitless (about `1.35`).
`--tg-text-padding`	The text inset of a message bubble, the `body` margin (about `8px 12px`).
`--tg-button-bg`, `--tg-button-text`, `--tg-button-radius`, `--tg-button-font`, `--tg-button-padding`, `--tg-button-height`, `--tg-button-pressed-opacity`, `--tg-button-disabled-opacity`	The app's button style, used by the default `<button>`.
`--tg-content-safe-area-inset-top`, `-right`, `-bottom`, `-left`	The band the app's time is drawn in: `bottom` is its height, the rest are 0. All are 0 with a caption below the widget. `env(safe-area-inset-*)` gives the same values.
`--tg-content-safe-area-corner-left`, `--tg-content-safe-area-corner-right`	The width the time takes in that band, on its side (`right` in a left-to-right interface). The other one is 0.

HapticFeedback

Field	Type	Description
impactOccurred(style)	Function	An impact: `light`, `medium`, `heavy`, `rigid` or `soft`.
notificationOccurred(type)	Function	A notification: `error`, `success` or `warning`.
selectionChanged()	Function	A selection change.

An unknown style or type throws `TypeError`. Calls play only while the widget is visible, at most 5 in a burst and 10 per second. They are free during a touch on the widget and for 5 seconds after it. Without a touch the first call opens a one-second series, and the next series may start after a touch or 10 seconds later. Other calls are dropped. Desktop and web apps play nothing.

ContentSafeAreaInset

Field	Type	Description
top, right, left	Number	0 in this version.
bottom	Number	The height of the band the app's time is drawn in at the bottom. 0 when the message has a caption below the widget.

Field	Type	Description
cornerRight	Number	The width the time takes at the right end of the band (a left-to-right interface), otherwise 0.
cornerLeft	Number	The width the time takes at the left end of the band (a right-to-left interface), otherwise 0.

WidgetViewer

The account looking at the widget, shaped like Mini Apps' `WebAppUser`, with the same fields the bot gets in `from` when this user taps. A field the app does not have, or that breaks its rule, is absent.

Field	Type	Description
id	Integer	The Telegram user id: the same id the bot gets as `from.id` with a request.
first_name	String	*Optional*. First name, 1-64 characters.
last_name	String	*Optional*. Last name, 1-64 characters.
username	String	*Optional*. Username, 5-32 characters of `A–Z`, `a–z`, `0–9` and `_`.
is_premium	Boolean	*Optional*. *True*, if the user has Telegram Premium.

WidgetRequestAnswer

Field	Type	Description
result	String	The bot's answer for this widget (the `data` of answerWidgetQuery), text of at most 1024 bytes, or `null` if the bot answered without one. Frozen.

WidgetWallet

See Payments with the wallet.

Field	Type	Description
sendTransaction(transaction, data)	Function	Asks the user to send `transaction`, TON Connect's `sendTransaction` request (`{ valid_until?, network?, from?, messages: [{ address, amount, payload?, stateInit? }] }`, 1-255 messages, a user-friendly `address`, `amount` in nanotons as a decimal string, `payload` and `stateInit` in base64), in the app's wallet window. `data`, 1-64 bytes, is the label the bot gets with the outcome. Returns a Promise that resolves with the bot's answer to the outcome, a WidgetRequestAnswer. A malformed transaction or a field outside TON Connect's set throws `TypeError`.

Field	Type	Description
signData(payload, data)	Function	Asks the user to sign `payload`, TON Connect's `signData` payload: `{ type: 'text', text }`, `{ type: 'binary', bytes }` or `{ type: 'cell', schema, cell }`. The signature goes to the bot with `data`, and the Promise resolves with the bot's answer.

Both may be called on a tap, or within 10 seconds of the answer to a call made on a tap. One call can be in flight, shared with requests.

WidgetRequestError

An `Error` with `name` set to `WidgetRequestError` and one of these values of `code`:

code	When
`NOT_ALLOWED`	This message may not ask: a widget sent without `allow_queries`, a forward of a static widget, a call outside a tap where background requests are off, a preview, an app that does not deliver requests, or a wallet call without a tap or a fresh answer.
`BUSY`	Another request or wallet call of this widget is in flight.
`RATE_LIMITED`	A background call less than 5 seconds after the last one, or the server's limit of 5 requests per 5 seconds per user and bot.
`TIMEOUT`	The bot did not answer in 15 seconds: it is down, does not receive `widget_query` and `shared_widget_query` updates, or is slow.
`FAILED`	Anything else: the bot was blocked or deleted, the message is gone, the network failed.
`INVALID_ANSWER`	The bot's answer broke the limits, so it never reaches the widget.
`REJECTED`	A wallet call: the user declined in the wallet window.
`BAD_REQUEST`	A wallet call: the wallet refused the transaction or the payload (another network or sender, an expired `valid_until`, cells it cannot parse).

Events available for widgets

eventType	Description
`themeChanged`	The chat's theme changed. `colorScheme`, `themeParams` and the CSS variables already have the new values.
`activated`	The widget became visible: scrolled into view, or the app came back to the foreground. Fires after `visibilitychange`.
`deactivated`	The widget is no longer visible. Fires after `visibilitychange`.
`chatDataChanged`	The bot changed the chat state. `chatData` already has the new state. Never fires in a forward.

eventType	Description
`publicDataChanged`	The bot changed the shared state. For a shared widget on screen, only while it watches (`watchPublicData()`). `publicData` already has the new state. Fires after `chatDataChanged` when one change touches both.
`contentSafeAreaChanged`	The part of the widget the app's time covers changed (a caption added, removed or moved by an edit, the “edited” mark). `contentSafeAreaInset` and the CSS variables already have the new values.

Testing

Counting the size

The server compresses the HTML with zlib at level 9 and counts the result. Python's zlib counts exactly as the server does:

```
python3 -c 'import sys,zlib; print(len(zlib.compress(sys.stdin.buffer.read(),
9)))' < widget.html
```

`gzip -9 -n` counts about 12 bytes more. Other zlib builds (Node.js's, for one) can be a few dozen bytes off either way, so near 4096 trust the Python number. If the bot gets `INPUT_MEDIA_WIDGET_TOO_LARGE` for HTML under 32 KB, it hit the compressed limit.

A first look in a browser

A widget is HTML, so a desktop browser with the window 320-430 px wide gives a quick first look at the layout. A browser is not a Telegram app: it draws everything, including what apps remove (What widgets support), enforces none of the limits, and has no `TelegramWidget`. For a local look, put a stand-in above the widget's own script, with the palette as variables, and remove both before sending:

```
<style>:root { --tg-theme-bg-color: #fff; --tg-theme-text-color: #000; --tg-
theme-hint-color: #707579;
  --tg-theme-link-color: #3390ec; --tg-theme-button-color: #3390ec; --tg-theme-
button-text-color: #fff;
  --tg-theme-secondary-bg-color: #f1f1f4; --tg-theme-destructive-text-color:
#e53935; }</style>
<script>
window.TelegramWidget = window.TelegramWidget || {
  profile: 1, isProfileAtLeast: (n) => n <= 1, colorScheme: 'light',
themeParams: {}, isActive: true,
  isPreview: false, isForward: false, chatType: 'private', viewer: { id: 1,
first_name: 'Test' },
  chatData: null, publicData: null, wallet: null,
```

```
  contentSafeAreaInset: { top: 0, right: 0, bottom: 0, left: 0, cornerLeft: 0,
cornerRight: 0 },
  HapticFeedback: { impactOccurred() { return this; }, notificationOccurred() {
return this; }, selectionChanged() { return this; } },
  onEvent() {}, offEvent() {}, ready() {}, setResizeAnchor() {},
watchPublicData() {}, unwatchPublicData() {},
  openLink: (url) => console.log('openLink', url), openTelegramLink: (url) =>
console.log('openTelegramLink', url),
  copyText: (text) => console.log('copyText', text),
  request: (data) => { console.log('request', data); return Promise.resolve({
result: null }); },
};
</script>
```

Set `chatData`, `publicData`, `viewer` and `isForward` to the cases you want to see, and answer `request` the way your bot would.

Testing in Telegram

Every place a message can go gives the widget different rights, so test each one with your bot. On the test server every bot has all of them. In the production environment a bot currently has only the private chat (Availability):

- **The private chat with the bot:** the chat state, background requests (one when the widget appears, then one per 5 seconds), a request made in the script's first lines.
- **A group with two accounts:** one chat state for everyone, `viewer` different in each account, requests on a tap only.
- **Inline mode:** the preview in the list of results, the message sent via the bot, its requests with `inline_message_id`.
- **Forwards:** a static widget's forward is a snapshot that cannot ask. A shared widget's forward follows the shared state and, if sent with `allow_queries`, asks through `shared_widget_query`.
- **The look:** both color schemes, an outgoing bubble (a widget sent via the bot), the largest text size, a narrow phone and a desktop app with a mouse, a message without a caption and with one above and below (the time in the corner), a right-to-left interface language.
- **The speed:** an older phone. The limits are the same everywhere, but a widget that is slow on an old phone stays slow there.

The test server also keeps experiments away from real users.

Writing widgets with an LLM

Models write widgets well when they are told the rules. Put this block into the model's system prompt:

```
You write Telegram widgets. A widget is one self-contained HTML document (or a
fragment) with inline <style> and <script>, sent by a bot in a message and
drawn
inside the chat bubble on every Telegram client (iOS, Android, desktop, web).
Every client silently removes what is outside the widget profile below and runs
the rest: an out feature is a bug you will not see. Docs:
https://core.telegram.org/bots/widgets.md

SIZE
- At most 32768 bytes of HTML and 4096 bytes after zlib level 9. The server
  refuses anything bigger. Python's zlib.compress(html, 9) counts as the server
  does. If you cannot measure, aim for 3800. Repeated CSS and markup compress
  well, data: images and long unique text do not. No comments, no dead code.

THE BUBBLE
- 200-600 px wide (phones about 320-430, so design for 320), as tall as the
  content, at most 600 px. The rest is clipped. Nothing scrolls inside a
  widget: overflow auto/scroll becomes hidden, position: sticky becomes
  relative. Fit long content into 600 px with pages, tabs or "show more".
- Lay out with %, flex, grid, vw/vmin. Never use vh (removed).
- html has no background: the bubble's color shows through, and a background
  on html or body paints the whole bubble. body has the client's text padding
  as its margin (about 8px 12px), and body{margin:0} goes edge to edge.
- Defaults: the client's message text (system-ui, --tg-text-font-size about
  15px, --tg-text-line-height) in --tg-theme-text-color. Headings, lists,
  tables and pre get native margins. An unstyled <button> looks like the
  client's own button. Any CSS of yours wins.
- Theme: CSS variables on the root, each '#rrggbb': --tg-theme-bg-color,
  --tg-theme-text-color, --tg-theme-hint-color, --tg-theme-link-color,
  --tg-theme-button-color, --tg-theme-button-text-color,
  --tg-theme-secondary-bg-color, --tg-theme-accent-text-color,
  --tg-theme-subtitle-text-color, --tg-theme-destructive-text-color,
  --tg-theme-section-bg-color, --tg-theme-section-header-text-color,
  --tg-theme-section-separator-color. --tg-color-scheme is light or dark.
  prefers-color-scheme, color-scheme and light-dark() follow the chat's theme,
  live. Build colors on these so both themes work.
- Fonts: system-ui, sans-serif, serif, monospace, ui-sans-serif, ui-serif,
  ui-monospace, ui-rounded only. Font names and @font-face are removed.
- Text is not selectable unless you set user-select:text and
  -webkit-user-select:text on what a user may want to copy.
- Unless a caption is below, the client draws the message time in the bottom
  corner. Keep env(safe-area-inset-bottom) clear at the bottom (the defaults
  pad an unstyled body, and if you pad for it yourself, set body{padding:0}).
  Only --tg-content-safe-area-corner-left|right of it is covered, so content
  may enter the band on the other side.
More: https://core.telegram.org/bots/widgets.md#designing-widgets

WHAT WORKS
```

- HTML: div, span, p, h1-h6, ul/ol/li, dl, table, details/summary (toggle event), figure, header/section/nav/article, b/i/em/strong/small/code/pre/mark/time/abbr, a, img (data: PNG, JPEG, GIF, WebP only), canvas, button.
- Inline SVG: svg, g, path, rect, circle, ellipse, line, polyline, polygon, text, tspan, defs, symbol, use, linearGradient, radialGradient, stop.
- CSS: flex, grid, positioning (fixed = inside the bubble), box model, borders, border-radius, box-shadow, opacity, linear/radial/conic gradients (and repeating), 2D transforms, filter and backdrop-filter with blur(), brightness() and drop-shadow() only, transitions, @keyframes, custom properties, calc/min/max/clamp, rgb()/hsl()/hex/light-dark() colors, ::before/::after, :hover (mouse only), :active, :not/:is/:where, structural pseudo-classes, [disabled] and :disabled on buttons, @media on width, aspect-ratio, orientation, prefers-color-scheme, prefers-reduced-motion, @supports, one-line text-overflow: ellipsis, touch-action, pointer-events.
- JS: ES2023 in an interpreter (no JIT: keep per-frame work small), Promises, async/await, setTimeout/setInterval, requestAnimationFrame, the DOM (createElement, querySelector, innerHTML, textContent, classList, style, dataset, addEventListener), MutationObserver, ResizeObserver, layout reads (getBoundingClientRect, getComputedStyle, offsetWidth, clientWidth, an inserted img's width: each is a round trip to the renderer, so cache them and never read them every frame), matchMedia, canvas 2D (paths, rect, roundRect, arc, fillText, measureText, gradients, setTransform, clip, drawImage of a data: image, with the canvas scaled by devicePixelRatio for sharp lines), new Image() with a data: src (load/error, then drawImage), Web Animations (el.animate), Intl and toLocale*String in the user's locale (navigator.language), crypto.getRandomValues, TextEncoder, URL, structuredClone, localStorage, sessionStorage, CSS.supports.
- Element ids are not global variables: use document.getElementById('x').

OUT OF THE PROFILE (removed by every client, while the rest of the widget still runs)
- HTML: ruby, rt, rp, bdi, bdo; Legacy presentational elements: center, font, big, tt, strike, nobr; <img src="data:image/svg+xml">; <img> with data: AVIF, BMP, ICO; <img src="https://…"> (any non-data: URL); Form controls: form, input (all types), select, option, optgroup, textarea, label, fieldset, legend, datalist, output; progress, meter (native gauges); dialog (show/showModal), popover attribute; audio, video, source, track; script (as DOM), iframe, frame, object, embed, link, meta, base, portal, template, slot, math, fencedframe, …; align, valign, bgcolor, border, cellpadding, cellspacing, color, face, nowrap; contenteditable, spellcheck, inputmode, enterkeyhint, autocapitalize; draggable (HTML drag and drop); tabindex, autofocus, accesskey (keyboard focus and shortcuts).
- SVG: clipPath, mask, pattern, marker; SVG filter primitives (fe*); SMIL animation: animate, animateTransform, animateMotion; foreignObject (HTML inside SVG); <image href="data:…"> inside SVG.
- CSS: display: flow-root, contents, list-item; Logical properties: margin-inline, padding-block, inset-inline, inline-size, …; Multi-column layout (columns, column-count, column-gap); position: sticky; overflow: auto / scroll (scrolling inside the widget); Colour functions: hwb(), lab(), lch(), oklab(),

```
oklch(), color(); color-mix(); background-clip: text (gradient text); url() /
image-set() with anything but a data: image or #fragment; font-family naming a
font ("Helvetica", "Courier New", "Comic Sans MS", …); @font-face (web fonts);
-webkit-line-clamp / line-clamp (multi-line ellipsis); text-shadow; writing-
mode, text-orientation (vertical text); 3D transforms: rotateX/Y/3d,
translateZ/3d, scaleZ/3d, matrix3d, perspective, transform-style, backface-
visibility; filter: grayscale(), sepia(), saturate(), contrast(), hue-rotate(),
invert(), opacity(); filter: url(#svg-filter); mix-blend-mode, isolation,
background-blend-mode; clip-path: inset(), circle(), ellipse(), polygon(); mask
/ -webkit-mask with gradients or data: images; vh / dvh / svh / vb: height-
relative units.
- Selectors: :has(); ::first-letter, ::first-line, ::marker; :focus, :focus-
visible, :focus-within; :checked, :disabled, :enabled, :valid, :invalid,
:placeholder-shown, ::placeholder; :host, :host-context(), ::slotted(),
::part(), :scope; :target.
- At-rules: @media (hover: hover), (pointer: fine|coarse); @media print,
resolution, other media features; @import, @namespace; @container queries
(container-type in the widget); @layer cascade layers; @property (typed custom
properties); @counter-style, @starting-style, @scope, @page, @font-feature-
values, @font-palette-values, @position-try; CSS nesting (.a { .b { } }, &).
- Animation: Magic move (shared-element morph between renders).
- Canvas: createConicGradient; createPattern from a canvas or image;
globalCompositeOperation (source-over, destination-out, multiply, …);
shadowBlur, shadowColor, shadowOffsetX/Y; ctx.filter (CSS filter on canvas
drawing); getImageData / putImageData / createImageData (pixel read-back);
toDataURL, toBlob, captureStream; WebGL, OffscreenCanvas, bitmaprenderer
contexts.
- JS: element.focus() / blur() / select(), document.activeElement: the widget
takes keyboard focus; Bundled libraries by CDN name: Chart.js 4, D3 7 (nothing
is fetched); fetch, XMLHttpRequest, WebSocket, EventSource, RTCPeerConnection,
sendBeacon, importScripts, Worker, BroadcastChannel; location assignment,
history, window.open (→ openLink), form submission; alert, confirm, prompt,
print; clipboard, share, vibrate, geolocation, mediaDevices, Notification;
attachShadow, customElements.define; Blob, FileReader, URL.createObjectURL;
requestFullscreen.
- TelegramWidget: TelegramWidget.sendMessage.
- Input: touchstart / touchmove / touchend (touch clients only); Key events:
keydown / keyup / keypress, beforeinput / input from keys, focus / blur /
focusin / focusout from keys; Enter / Space activate a focused button or link
(a click from the keyboard); A key press counts as a user gesture for
privileged bridge calls; wheel and scroll events.
More: https://core.telegram.org/bots/widgets.md#what-widgets-support

INPUT
- Taps (click) and pointer events (pointerdown/move/up/cancel, enter/leave,
  setPointerCapture), with the mouse events (a coalesced move sums
  movementX/Y). No touch events (use pointer events), no keyboard, no focus, no
  text input, no wheel or scroll events.
- A double tap is always the widget's: two clicks, then dblclick.
```

```
- Vertical drags always scroll the chat: nothing can be dragged up or down (no
  vertical sliders, wheel pickers, joysticks, drawing or map panning). Use
  horizontal controls or buttons.
- Holds and horizontal drags are claimed per element with CSS touch-action,
  which covers the element's children: auto (the default): a hold opens the
  chat's message menu, a sideways swipe is the chat's reply, and the widget
  gets pointercancel and no click. pan-y: the widget gets horizontal drags
  (sliders, carousels, paddles). none: horizontal drags and holds, plus
  contextmenu (held game buttons). pan-x claims nothing. Keep auto everywhere
  else. On pointercancel undo what pointerdown started.
- The leading 20 px never take a claim (the system back gesture): keep
  controls 20 px from the left edge (the right in RTL).
- A hold is 500 ms within 10 px, and a double tap is two taps within 300 ms
  and 40 px.
More: https://core.telegram.org/bots/widgets.md#gestures

TELEGRAMWIDGET (window.TelegramWidget, undefined outside Telegram, so check it)
- profile (1), isProfileAtLeast(n). colorScheme: 'light' | 'dark'.
  themeParams: { bg_color, text_color, hint_color, ... } ('#rrggbb', the keys
of
  the CSS variables with _ for -).
- onEvent(type, fn) / offEvent(type, fn), type: 'themeChanged', 'activated' /
  'deactivated' (on and off screen, isActive), 'chatDataChanged',
  'publicDataChanged'.
- ready(): optional, once the first state is drawn.
- openLink(url): https only, at most 256 characters. The client shows the full
  URL and the user confirms. openTelegramLink(url): https://t.me/..., opens
  inside Telegram without a confirmation, as a t.me link in a message. A tapped
  <a href="https://..."> does the same. A bad URL throws TypeError.
- copyText(text): 1-256 characters. The client copies it and shows a toast.
- openLink, openTelegramLink, copyText and request need a real tap: call them
  synchronously in a click (or pointerup) handler, not after await, a timer or
  a promise, nor in a finger's pointerdown (it may still become the chat's
  scroll). One of them per tap. Otherwise they are refused.
- HapticFeedback.impactOccurred('light'|'medium'|'heavy'|'rigid'|'soft'),
  .notificationOccurred('error'|'success'|'warning'), .selectionChanged():
  only on screen, free during a touch and 5 s after it. Without a touch, one
  short series, then nothing for 10 s (a timer cannot buzz the phone).
- setResizeAnchor('top' | 'bottom'): the edge of the bubble that stays still
  when your height changes ('top' by default, 'bottom' for a log or thread
  that grows at the bottom). Older clients lack it:
  TelegramWidget.setResizeAnchor?.('bottom').
- No sendMessage, alert/confirm/prompt, navigator.clipboard (use copyText),
  window.open (use openLink), share or fullscreen.
More: https://core.telegram.org/bots/widgets.md#telegramwidget

THE WIDGET'S STATE AND REQUESTS (only the bot writes the state)
- The HTML never changes after sending, and the bot changes only the state.
  chatData and publicData are strings of up to 1024 bytes, or null:
```

```
  JSON.parse them yourself (in try/catch) and draw from them.
  chatData is the chat state: this message in its chat, the same for every
  member of a group (a board, whose turn). publicData is the shared state,
  the same wherever the widget is shown, forwards included (poll results).
  A change of either fires chatDataChanged / publicDataChanged. Redraw from
  it, nothing restarts.
- watchPublicData(): the client polls the shared state live while the widget
  is on screen (a ticker, a poll others vote in). unwatchPublicData() stops.
  A poll may bring a state older than a result gave you: keep a revision in
  the state (the bot raises it per edit and returns it in result) and ignore
  older ones.
- request(data): ask the bot, like a callback button. data is 1-64 bytes.
  Resolves with { result }: the bot's text for this user (up to 1024 bytes) or
  null. Rejects with an error whose code is 'NOT_ALLOWED' (no tap, requests not
  allowed here, a forward, a preview), 'BUSY' (one request at a time),
  'RATE_LIMITED', 'TIMEOUT', 'FAILED' or 'INVALID_ANSWER'. Show a pending state
  and every failure. The answer does not change the state: the bot's edit
  arrives as the change events. For the new state at once, have the bot
  return it in result.
- Without a tap request works only in the private chat with the bot: once
  when the widget appears on screen, then at most once per 5 s while visible.
  One made off screen (at start: the client creates the bubble before showing
  it) waits and goes out when the widget appears, so ask in the first lines.
- viewer: { id, first_name, last_name?, username?, is_premium? } of the person
  looking, or null where the widget may not ask the bot. Compare viewer.id
  with the ids in the state (my turn, my vote).
- isForward: a forwarded copy. chatData is null and chatType too. It may ask
  only if the bot allowed forwards to (otherwise viewer is null and every
  request is NOT_ALLOWED), so show the shared state read-only. A forward of a
  widget that is not shared never changes: draw a snapshot ("as of 19:40"),
  not live progress.
- chatType: 'private' | 'group' | 'supergroup' | 'channel', or null.
- isPreview: shown in the bot's inline results before it is sent: no taps, no
  requests, links, copy or storage, viewer null. Draw the state as it will be
  sent.
- wallet (Telegram's Gram Wallet, TON Connect's formats): null where the
  client has no wallet for this user, the widget may not ask the bot, or in a
  preview. To pay, on a tap: request('pay:1', { wallet: true }) gives the bot
  the user's wallet address. The bot answers with a fresh transaction
  { valid_until, messages: [{ address, amount, payload?, stateInit? }] } (1-255
  messages, address user-friendly, amount: nanotons as a decimal string). In
  the answer's handler, wallet.sendTransaction(tx, 'paid:1') needs no new tap
  (within 10 s). The client's window shows the bot and amounts, and the
  promise resolves with the bot's answer { result }.
  wallet.signData({ type: 'text', text }, data) asks for a signature, which
  goes to the bot. One wallet call per tap or answer, never from a timer.
  Codes: 'REJECTED' (declined), 'BAD_REQUEST' (the wallet refused it). A
  malformed transaction throws TypeError. After TIMEOUT ask the bot, do not
  pay again.
```

```
More: https://core.telegram.org/bots/widgets.md#the-widgets-state

STORAGE, LIFECYCLE, LIMITS
- localStorage: this message on this device, 4 KB (QuotaExceededError past
  it, so catch it). sessionStorage lasts until the app restarts. A forward
  starts empty. Keep this user's own choices there (best score, "voted").
  What everyone sees belongs in the bot's state.
- The widget can be destroyed and recreated at any time: restore from
  localStorage and the state on load, and draw the first frame synchronously.
- Off screen: document.hidden, no animation frames, timers at most 1 Hz.
- CPU: 300 ms per second (timers and frames wait beyond it, longer after a
  task that ran past it). One task may take 500 ms, longer stops the widget.
  At most 128 layout reads in one task. They plus about 25 µs per DOM change
  cost the client at most 100 ms per second (frames wait beyond it).
- Memory 8 MB. 64 live timers and 4000 canvas commands per canvas per task
  (RangeError past them). A data: image is at most 1024 x 1024 pixels
  (larger: a placeholder). The widget stops on its last frame ("Widget
  stopped") past the CPU or memory limit, 2000 nodes, one text value of 32 K
  characters, 48 K of text in all, an attribute of 8 K, 10000 DOM changes and
  method calls per task, 400 CSS rules, 32 @keyframes, 64 script animations
  or 2048 x 2048 canvas pixels together.
- At most 16 elements at once with a blurred box-shadow, filter,
  backdrop-filter or opacity below 1 (animated ones and ::before/::after
  too), 2 of them with backdrop-filter. Past it the widget stops. Mute with
  color (var(--tg-theme-hint-color), rgba(), fill-opacity), never opacity. A
  shadow without blur is free.
- Animate with CSS or requestAnimationFrame. Change only what moved, and do
  not rebuild the DOM every frame.

EXAMPLE (the patterns, not a template)
<style>
body{margin:0;padding:12px 14px}
.opt{display:block;width:100%;margin:6px 0;text-align:left}
.me{outline:2px solid var(--tg-theme-button-color)}
.bar{height:4px;margin-top:4px;border-radius:2px;background:var(--tg-theme-
button-color)}
.hint{color:var(--tg-theme-hint-color);font-size:13px}
</style>
<div id="list"></div><div class="hint" id="msg"></div>
<script>
const TW = window.TelegramWidget, $ = (id) => document.getElementById(id);
const NAMES = ['Pizza', 'Sushi'];
const votes = () => { try { return JSON.parse(TW && TW.publicData).v; } catch {
return [0, 0]; } };
let mine = null; try { mine = JSON.parse(localStorage.getItem('vote')); } catch
{}
function draw() {
  const v = votes(), total = v.reduce((a, b) => a + b, 0) || 1;
  $('list').replaceChildren(...NAMES.map((name, i) => {
```

```
    const b = document.createElement('button'), pct = Math.round(v[i] / total *
100);
    b.className = 'opt' + (mine === i ? ' me' : '');
    b.innerHTML = name + ' <span class="hint">' + pct + '%</span><div
class="bar" style="width:' + pct + '%"></div>';
    b.onclick = () => vote(i);
    return b;
  }));
}
function vote(i) {
  if (!TW) return;
  $('msg').textContent = 'Sending...';
  TW.request('vote:' + i).then(() => {
    mine = i; try { localStorage.setItem('vote', i); } catch {}
    $('msg').textContent = ''; draw();
  }, (e) => { $('msg').textContent = e.code === 'NOT_ALLOWED' ? 'Open the bot
to vote' : 'Not sent: ' + e.code; });
}
if (TW) TW.onEvent('publicDataChanged', draw);
draw();
</script>
```

Then give the model your bot's side of the contract:

- **The protocol**: the JSON shapes of the chat state, the shared state and the answers, the request `data` values the bot understands, and how you send the widget (static or `is_shared`, with or without `allow_queries`).
- **Live shared state**: if the widget shows a shared widget's shared state that the bot keeps changing (counts, prices, scores), it calls `TelegramWidget.watchPublicData()` and keeps the newest revision. Otherwise it gets a new shared state only with the message.
- **The places**: what the widget should show in a forward (no chat state, `chatType` `null`, maybe no requests), in a group (one chat state for everyone, drawn per `viewer.id`) and in a preview (`isPreview`: no taps).
- **The checks**: have the model count the size as in Counting the size, compare its widget with the out list in the block, and test the result in Telegram before you send it to users.